

Explicação didática — menu02.c

Sistema de Gestão de Clientes (UNICIVE) — Disciplina de Estrutura de Dados
Autor original: Aguinaldo Alves · Documento de estudo gerado para revisão do código

Objetivo deste documento: explicar, parte por parte, o que cada trecho do código `menu02.c` faz e por que ele foi escrito daquela forma. A ideia é que você consiga acompanhar o raciocínio do programa como se estivesse lendo-o pela primeira vez, entendendo tanto a sintaxe da linguagem C quanto a lógica de um cadastro de clientes usando um vetor de *structs*.

1. Visão geral do programa

O programa implementa um **CRUD** (Create, Read, Update, Delete — incluir, consultar, alterar e excluir) de clientes, todo guardado em memória (um vetor), sem gravar em arquivo nem em banco de dados. Enquanto o programa está rodando, os clientes cadastrados ficam guardados numa estrutura chamada `tipo_lista`; quando o programa é fechado, tudo se perde, porque nada é salvo em disco.

A interface é toda feita no modo texto do console do Windows, usando a função `gotoxy` para posicionar o cursor em coordenadas (x, y) e desenhar uma "telinha" com bordas, dando a impressão de um formulário.

2. Cabeçalho e bibliotecas incluídas

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <windows.h>
```

Biblioteca	Para que serve aqui
<code>stdio.h</code>	Entrada e saída padrão: <code>printf</code> , <code>scanf</code> , <code>fgets</code> .
<code>stdlib.h</code>	Função <code>system("cls")</code> , que limpa a tela do console.
<code>conio.h</code>	Função <code>getch()</code> , que lê uma tecla sem precisar de Enter (pausa a tela).
<code>windows.h</code>	Tipos e funções específicos do Windows, como <code>COORD</code> e <code>SetConsoleCursorPosition</code> , usados para mover o cursor.

Ponto de atenção: por depender de `windows.h` e `conio.h`, este programa só compila e roda no Windows. Em Linux/Mac, essas bibliotecas não existem — o código não é portátil.

3. Constantes

```
#define INICIO_ARRANJO 1
#define TAMANHO_MAXIMO 10
```

`#define` cria uma **constante de pré-processador**: antes de compilar, o compilador substitui todo `TAMANHO_MAXIMO` por `10` no texto do código. Isso evita "números mágicos" espalhados pelo programa — se um dia você quiser um cadastro para 50 clientes, basta mudar esse número em um único lugar.

Observação: a constante `INICIO_ARRANJO` é declarada mas nunca é usada em nenhum lugar do código. Isso não gera erro de compilação, só fica "sobrando".

4. Estruturas de dados (structs)

4.1 — `reg_cliente`: os dados de UM cliente

```
typedef struct
{
    int cd_cliente;
    char nm_cliente[50];
    char ds_endereco[50];
    int nr_numero;
    char nr_documento[20];
    char ds_cidade[50];
    char ds_uf[05];
    char dt_cadastro[19];
    char nr_telefone[15];
} reg_cliente;
```

Uma `struct` agrupa vários dados diferentes sob um único "pacote". Aqui, cada cliente tem um código numérico, nome, endereço, número da casa, documento, cidade, UF, data de cadastro e telefone — tudo isso vira **um único tipo de dado** chamado `reg_cliente`.

O `typedef` antes do `struct` serve para dar um "apelido" a esse tipo. Sem o `typedef`, toda vez que fôssemos declarar uma variável desse tipo teríamos que escrever `struct reg_cliente x;`. Com o `typedef`, basta escrever `reg_cliente x;`.

Conceito-chave — vetores de char como texto: em C não existe um tipo "string" pronto. Um texto é representado como um vetor de caracteres (`char nome[50]`), terminado por um caractere especial `'\0'` (o "fim de string"). É por isso que campos como `ds_uf[05]` (5 posições) só cabem, na prática, 4 letras + o `\0` final.

4.2 — tipo_lista: o "banco de dados" em memória

```
typedef struct
{
    reg_cliente dados[TAMANHO_MAXIMO];
    int primeiro;
    int ultimo;
} tipo_lista;
```

Esta segunda struct representa a **lista de clientes inteira**. Ela contém:

- `dados[TAMANHO_MAXIMO]` — um vetor com espaço para 10 clientes (cada posição é um `reg_cliente` completo).
- `primeiro` — declarado, mas nunca de fato utilizado na lógica do programa.
- `ultimo` — funciona como um **contador**: guarda quantos clientes já foram cadastrados e, ao mesmo tempo, indica a próxima posição livre do vetor. Por exemplo, se `ultimo == 3`, significa que existem clientes nas posições 0, 1 e 2, e o próximo cliente entrará na posição 3.

5. Funções auxiliares de tela

5.1 — gotoxy(x, y)

```
void gotoxy(int x, int y)
{
    COORD coord;
    coord.X = x;
    coord.Y = y;
    SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), coord);
}
```

Esta função move o "cursor de escrita" do console para a coluna `x` e linha `y`, usando funções do Windows. É graças a ela que o programa consegue desenhar rótulos e campos em posições fixas da tela, como um formulário, em vez de simplesmente empilhar textos um embaixo do outro.

5.2 — telabordas(): desenha a moldura da tela

Limpa a tela (`system("cls")`) e desenha, usando `printf` combinado com `gotoxy`, as bordas de uma "janela" de texto: uma linha de `+++++` no topo, o título "UNICIVE — SISTEMA DE GESTÃO DE CLIENTES — Estrutura de Dados", linhas laterais com `|` repetidas de 5 a 21 (usando um `for`), e o fechamento embaixo. É a base visual reutilizada por todas as outras telas do sistema.

5.3 — telacliente(): rótulos do formulário do cliente

Chama `telabordas()` e depois escreve, em posições fixas, os rótulos dos 8 campos do cliente (Nome, Endereço, Número, CPF, Cidade, Estado, Data Cadastro, Telefone) mais o rótulo do código. Repare que ela só desenha os **rótulos** — quem preenche os valores é a função `exibir_cliente` (quando o cliente já existe) ou o próprio `scanf` / `fgets` (quando o usuário está digitando).

6. menu(): exibição e leitura da opção

```
int menu(int opcao)
{
    telabordas();
    ...
    printf("Escolha uma opcao: ");
    scanf("%d", &opcao);
    return opcao;
}
```

Desenha as bordas e as opções (1 a 4, e 0 para sair), lê a opção digitada pelo usuário com `scanf` e **retorna** esse valor para quem chamou a função.

Detalhe curioso: a função recebe um parâmetro `opcao`, mas nunca usa o valor recebido — ela sobrescreve `opcao` com o que o `scanf` ler. Ou seja, o parâmetro de entrada é inútil; só o valor de retorno importa. Isso funciona, mas é um sinal de que o parâmetro poderia ser removido sem alterar o comportamento do programa.

7. limpar_mensagem(): "apagar" uma linha da tela

Como o console não tem como apagar só um texto (ele não é uma interface gráfica), esta função "apaga" visualmente a linha de mensagens (linha 23) posicionando o cursor lá e escrevendo um monte de espaços em branco por cima do texto anterior. É uma técnica simples e comum em telas de console.

8. pesquisa(): busca sequencial na lista

```
int pesquisa(tipo_lista *L, int codigo)
{
    int x;
    for (x = 0; x < L->ultimo; x++)
    {
        if (L->dados[x].cd_cliente == codigo)
        {
            return x;
        }
    }
    return -1;
}
```

Esta é uma das funções mais importantes do programa, pois é reutilizada em `incluir`, `alterar`, `excluir` e `consultar`. Ela percorre o vetor `dados`, posição por posição, do índice 0 até `ultimo - 1`, comparando o `cd_cliente` de cada posição com o `codigo` procurado.

- Se encontrar, retorna o **índice** (a posição) onde o cliente está.
- Se percorrer tudo e não encontrar, retorna `-1`, um valor "sentinela" que sinaliza "não encontrado".

Conceito-chave — ponteiro para struct (`tipo_lista *L`): em vez de receber a lista inteira "por cópia" (o que seria caro em memória e não permitiria alterações), a função recebe o **endereço** da lista original, através de um ponteiro. Por isso se usa a seta `L->ultimo` e `L->dados[x]` em vez do ponto (`.`): a seta é um atalho para "primeiro pegar o valor apontado por `L` , depois acessar o campo".

Complexidade: como essa busca olha um elemento de cada vez até achar (ou até acabar o vetor), ela é chamada de **busca sequencial (ou linear)**, com complexidade $O(n)$. Com apenas 10 clientes isso é irrelevante, mas é o tipo de estrutura que, em listas grandes, pode ser otimizado com buscas mais eficientes (ex.: busca binária em vetor ordenado, ou uma árvore).

9. `exibir_cliente()`: mostra os dados de um cliente na tela

```
void exibir_cliente(reg_cliente clie)
{
    gotoxy(28, 05);
    printf("%d", clie.cd_cliente);
    ...
}
```

Recebe um `reg_cliente` **por valor** (ou seja, recebe uma *cópia* da struct, não o original) e, usando `gotoxy` + `printf` , escreve cada campo na posição correspondente ao rótulo já desenhado por `telacliente()` . É usada por `alterar` , `excluir` e `consultar` para exibir o cliente encontrado antes de qualquer ação.

10. incluir(): cadastrar um novo cliente

Esta função é a mais longa do programa. Ela é organizada em dois laços `do...while` aninhados:

1. **Laço interno** — pede o código do cliente repetidamente até que o usuário digite `0` (sair) ou um código que **ainda não existe** na lista (usando `pesquisa()` para checar). Se o código já existir, mostra "Código já cadastrado..." e pede de novo.
2. **Preenchimento dos dados** — se o código digitado não for `0`, o programa lê os demais campos (nome, endereço, número, CPF, cidade, UF, data, telefone) usando `scanf` (para o número) e `fgets` (para os textos).
3. **Confirmação e gravação** — pergunta se o usuário quer realmente gravar (`1 = Sim`). Se sim, verifica se a lista já está cheia (`L->ultimo >= TAMANHO_MAXIMO`); se não estiver cheia, grava o cliente na posição `L->ultimo` e incrementa o contador.
4. **Laço externo** — pergunta se o usuário quer cadastrar outro cliente, repetindo todo o processo enquanto a resposta for `1`.

Por que `fflush(stdin)` antes de cada `fgets` ?

Quando se mistura `scanf("%d", ...)` com leitura de texto, o Enter digitado depois do número fica "sobrando" no buffer de entrada. Se não for descartado, o próximo `fgets` lê apenas essa quebra de linha "fantasma" em vez de esperar o texto do usuário. O programa usa `fflush(stdin)` para limpar esse buffer.

Ponto de atenção (para estudo): o padrão da linguagem C não garante o comportamento de `fflush()` para a *entrada* (`stdin`) — oficialmente, `fflush` foi criado para limpar buffers de *saída*. Na prática, compiladores como o do Windows/MinGW costumam aceitar esse uso e ele "funciona", mas é considerado **comportamento não padronizado**. Vale saber disso para quando for programar em outros compiladores/sistemas, onde essa técnica pode não funcionar do mesmo jeito.

11. alterar(): editar um cliente existente

Segue uma lógica parecida com `incluir()`, mas em vez de checar se o código **não existe**, ela checa se o código **existe** (com `pesquisa()`). Passos:

1. Pede o código do cliente a alterar.
2. Se o código não for encontrado (e for diferente de 0), avisa "Cliente não encontrado..." e pausa com `getch()`.
3. Se for encontrado, primeiro **mostra os dados atuais** com `exibir_cliente(L->dados[posicao])`, para o usuário ver o que já está cadastrado.
4. Depois pede que o usuário digite **todos os campos novamente** (não permite editar só um campo específico), sobrescrevendo a struct temporária `clie`.
5. Pergunta se quer confirmar a gravação; se sim, substitui a posição encontrada: `L->dados[posicao] = clie;`. O código do cliente (`cd_cliente`) acaba sendo o mesmo, pois é o mesmo valor buscado antes.

12. excluir(): remover um cliente da lista

```
if (resp == 1)
{
    for (x = posicao; x < L->ultimo - 1; x++)
    {
        L->dados[x] = L->dados[x + 1];
    }
    L->ultimo--;
}
```

Depois de localizar o cliente com `pesquisa()`, mostrá-lo e confirmar a exclusão, o programa não "apaga" a posição — ele **desloca todos os elementos seguintes uma posição para trás**, sobrescrevendo o cliente excluído, e por fim diminui o contador `ultimo` em 1.

Por que deslocar em vez de só marcar como vazio? Como a busca (`pesquisa`) e o cadastro de novos clientes sempre assumem que os dados válidos vão de `0` até `ultimo - 1` sem "buracos", deslocar os elementos mantém essa regra simples. A desvantagem é o custo: em uma lista de `n` elementos, excluir do início pode exigir mover até `n-1` posições.

13. consultar(): apenas visualizar um cliente

É a função mais simples das quatro operações: pede um código, pesquisa, e se encontrar, apenas chama `exibir_cliente()` para mostrar os dados — sem alterar nada. Se não encontrar, avisa "Cliente não encontrado...".

14. main(): o coração do programa

```
int main()
{
    int opcao;
    tipo_lista L;

    L.primeiro = 0;
    L.ultimo = 0;

    do
    {
        system("cls");
        opcao = menu(opcao);

        switch (opcao)
        {
            case 1: incluir(&L); break;
            case 2: alterar(&L); break;
            case 3: excluir(&L); break;
            case 4: consultar(&L); break;
            case 0: printf("FINALIZAR O PROGRAMA \n"); break;
            default: printf("OPCAO INVALIDA \n"); break;
        }

    } while (opcao != 0);

    return 0;
}
```

A função `main()` é o ponto de entrada do programa. Ela:

1. Declara a variável `L`, do tipo `tipo_lista` — **essa é a única instância da lista de clientes, criada uma única vez**, fora do laço, para que os clientes cadastrados não se percam a cada volta do menu.
2. Zera `L.primeiro` e `L.ultimo` antes do laço começar.
3. Entra em um laço `do...while` que se repete enquanto a opção escolhida for diferente de `0`.
4. A cada volta, chama `menu()` para mostrar as opções e capturar a escolha do usuário.
5. Usa um `switch` para decidir qual função chamar de acordo com a opção — sempre passando o **endereço** de `L` (`&L`), para que as funções possam alterar a lista original.

Por que passar `&L` (endereço) e não `L` (a struct inteira)? Se as funções `incluir`, `alterar`, etc. recebessem `L` por valor, elas trabalhariam sobre uma *cópia* da lista, e qualquer inclusão/alteração feita dentro delas se perderia assim que a função terminasse — o `main` nunca veria a mudança. Passando o endereço (`&L`) e recebendo um *ponteiro* (`tipo_lista *L`), as funções operam diretamente sobre a lista original.

15. Resumo dos conceitos de Estrutura de Dados aplicados

Conceito	Onde aparece no código
Struct / registro	<code>reg_cliente</code> — agrupa vários campos de um mesmo cliente.
Vetor (array) como lista	<code>reg_cliente dados[TAMANHO_MAXIMO]</code> dentro de <code>tipo_lista</code> .
Contador de elementos válidos	<code>L->ultimo</code> , que também indica a próxima posição livre.
Ponteiros e passagem por referência	Todas as funções que recebem <code>tipo_lista *L</code> e usam <code>&L</code> no <code>main</code> .
Busca sequencial (O(n))	Função <code>pesquisa()</code> .
Inserção no fim do vetor	<code>L->dados[L->ultimo] = clie; L->ultimo++;</code> em <code>incluir()</code> .
Remoção com deslocamento	Laço <code>for</code> em <code>excluir()</code> que "puxa" os elementos seguintes.
Modularização	Cada operação do CRUD é isolada em sua própria função.

16. Pontos para você refletir / estudar mais

- O que aconteceria se você tentasse incluir um 11º cliente? (Veja a checagem `L->ultimo >= TAMANHO_MAXIMO` em `incluir()`.)
- Por que a função `pesquisa()` recebe um ponteiro (`tipo_lista *L`) mas a função `exibir_cliente()` recebe a struct por valor (`reg_cliente clie`)? Pense em quem precisa *alterar* os dados originais e quem só precisa *ler*.
- Como você adaptaria o programa para salvar os clientes em um arquivo (`.txt` ou binário), para que os dados não se percam ao fechar o programa?
- O que muda na função `excluir()` se, em vez de deslocar os elementos, você apenas "marcasse" a posição como vazia? Quais funções precisariam mudar para lidar com essas posições vazias?

Documento gerado automaticamente a partir da leitura do arquivo `menu02.C` para fins de estudo. As observações marcadas como "ponto de atenção" são comentários didáticos sobre boas práticas em C, não correções obrigatórias do código.